

JAVACREAM

*Training
Consulting
Projectmanagement*

Prometheus und Grafana



- Name
- Rolle im Unternehmen
- Themenbezogene Vorkenntnisse
- Aktuelle Problemsituation
- Individuelle Zielsetzung

- <https://cegos.deskmate.me/>
- Authentifizierung mit
 - tn0xy.raum01@cegos.de
 - 33155_tn0xy
- Anmeldung bei Ubuntu
 - sl01
 - sl01

xy: Eindeutige, zweiziffrige Zahl im Wertebereich 16-28

sl01 ist in der sudoer Gruppe

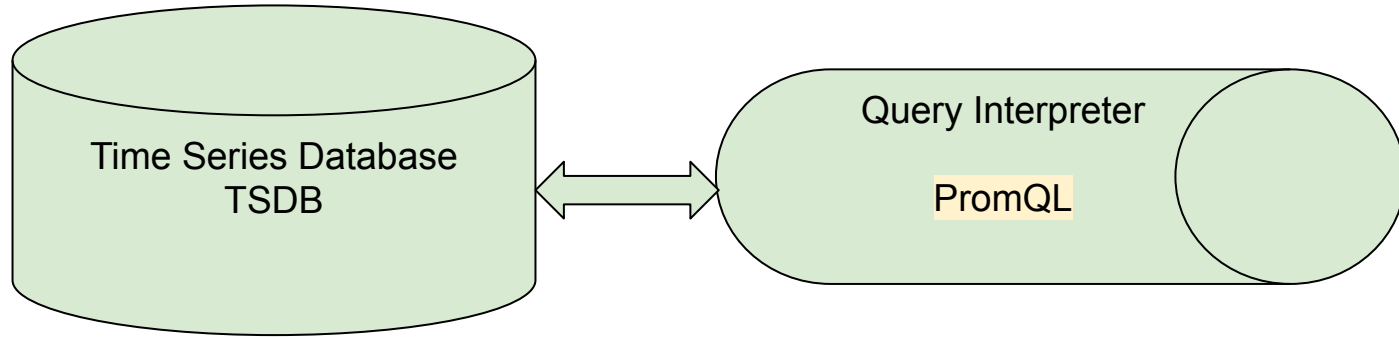
- tn018.raum01@cegos.de
- 33155_tn018

Ausgangssituation

- Anwendungsentwicklung Nullkommanull
 - Kein Debugging
 - Kein Optimieren und Tuning
 - Kein Testwerkzeug
- Logging: Nein
 - Request-Log / Transaction Log wird in Prometheus nicht erfasst
 - Hinweis: Hier wäre eine Grafana / Loki-Kombination möglich
- Erfassung von Metriken in Produktion ist der Primärfokus von Prometheus
 - Eine Metrik hat
 - Einen Zahlenwert
 - Eine "physikalische" Einheit
 - Identifier mit einem sprechenden Namen sowie gegebenenfalls Label / Tags
 - Zeitstempel

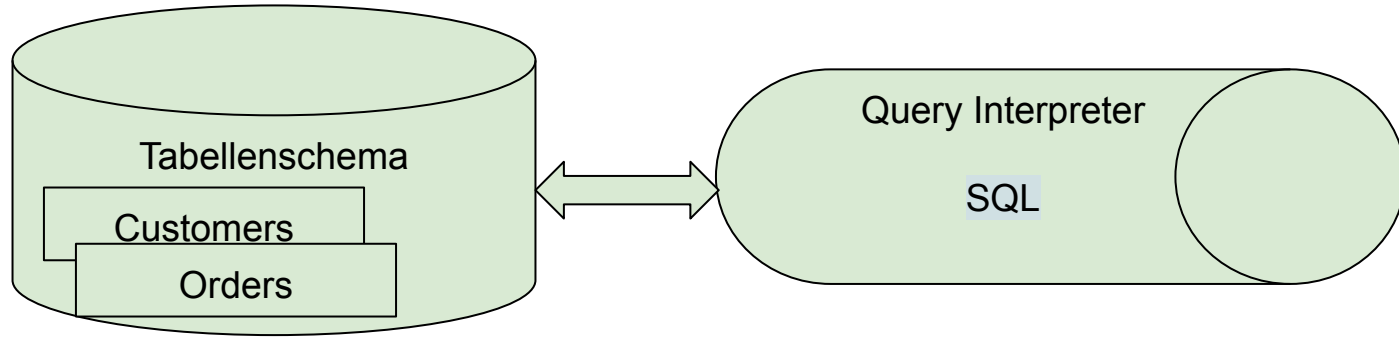
- Erfassen der Metriken
 - aus einer beliebigen Anzahl von Systemen
- Persistente Ablage der Metriken
- Analysewerkzeuge für Metriken im wesentlichen statistischen Verfahren
 - Bezug zur Mathematik
 - Machine Learning / KI durch externe Tools möglich, noch nicht Bestandteil von Prometheus Core

Prometheus



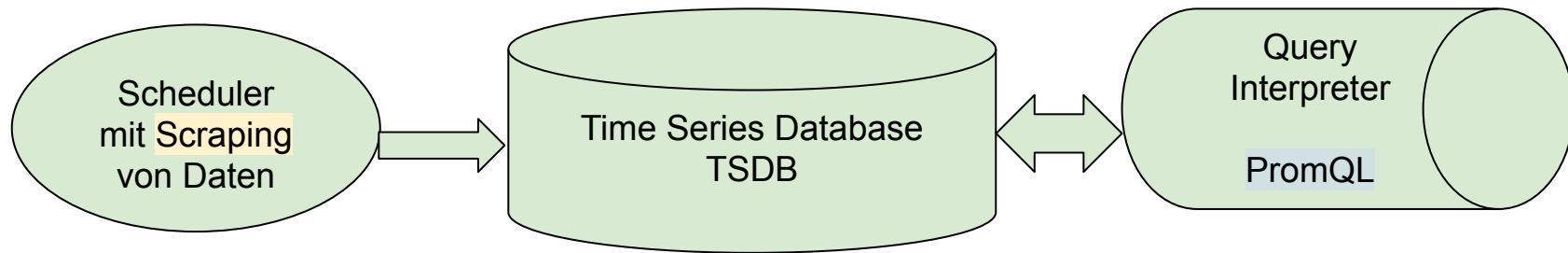
PromQL stellt Abfragen und statistische Funktionen zur Verfügung

z.B. MySql



SQL stellt Abfragen zur Verfügung, z.B.
"Zeige alle Bestellungen eines Kunden"

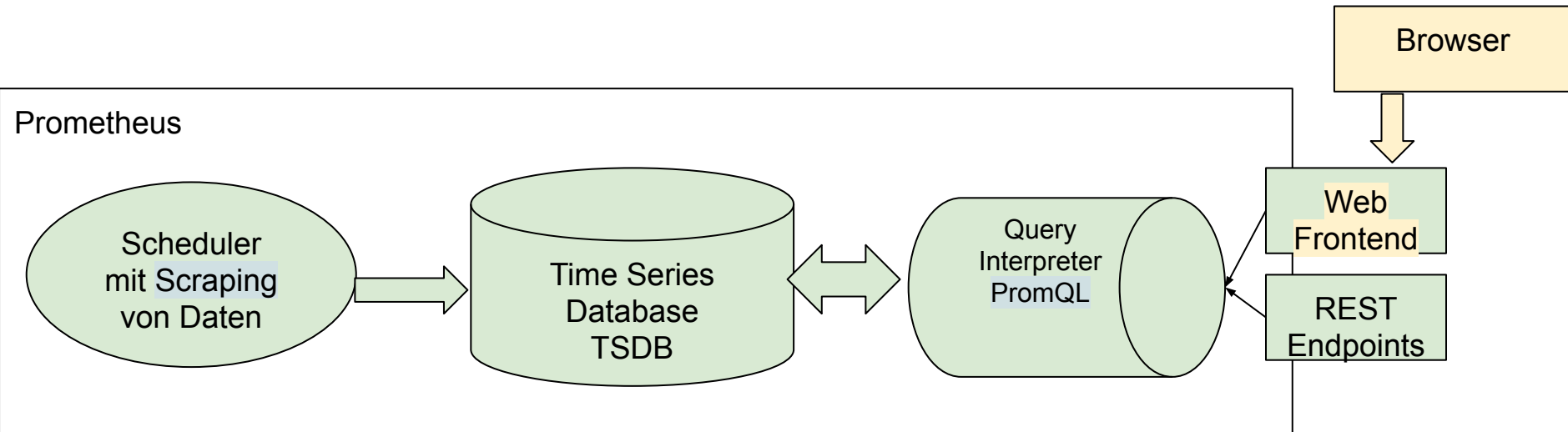
Prometheus



Scraping ist ein Lesen einer textuellen Information, die im Prometheus-Format bereitgestellt wird von einem http-Endpoint

PromQL stellt Abfragen und statistische Funktionen zur Verfügung

Der Prometheus Server



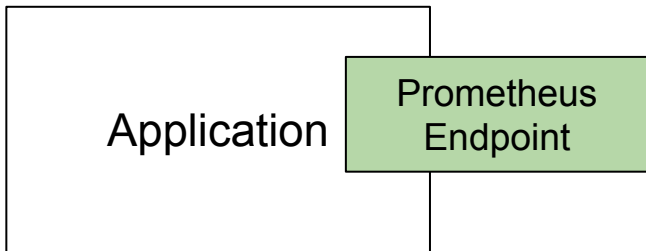
Scraping ist ein Lesen einer textuellen Information, die im Prometheus-Format bereitgestellt wird von einem http-Endpoint

PromQL stellt Abfragen und statistische Funktionen zur Verfügung

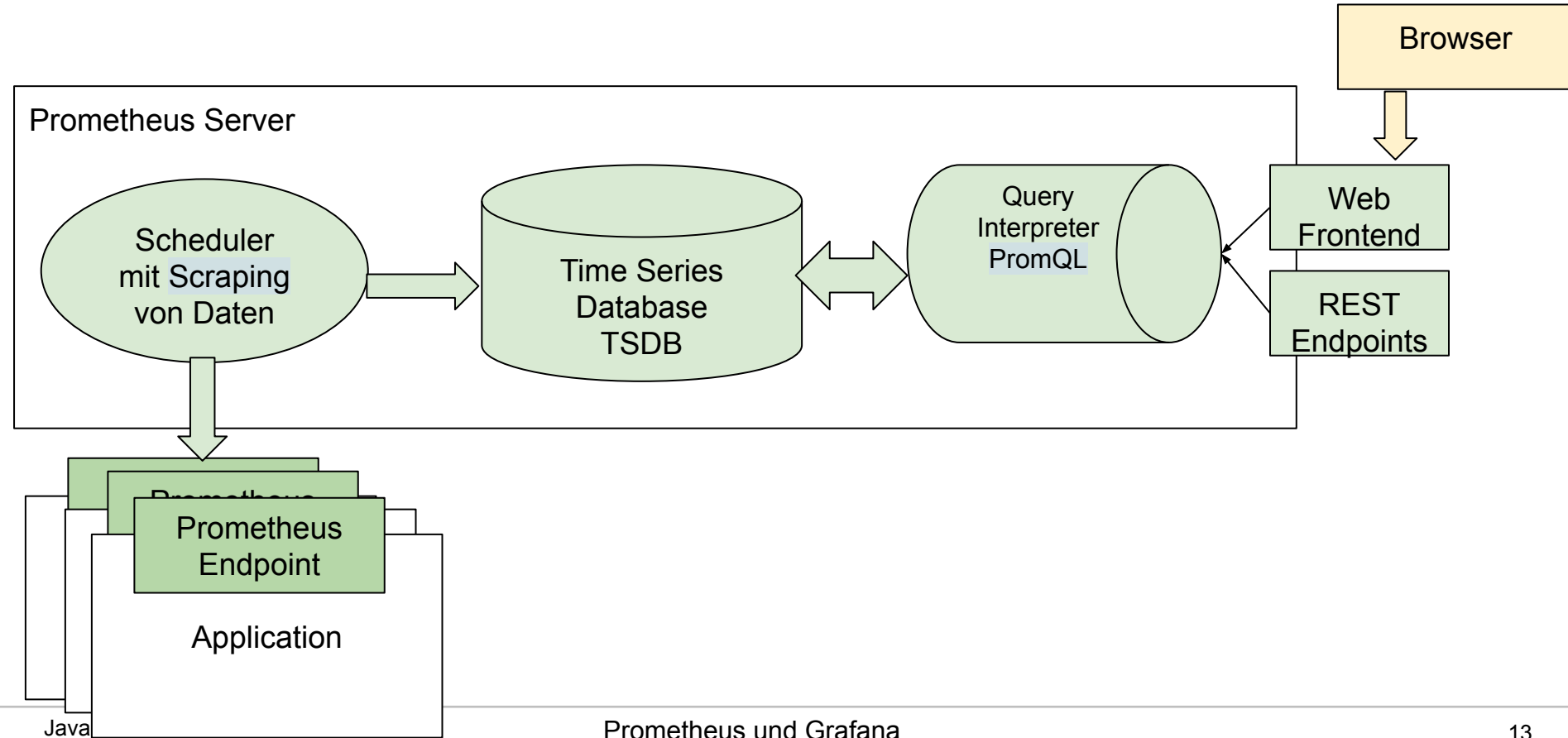
(Rudimentäre) Benutzeroberfläche

- Woher kommen die Daten?

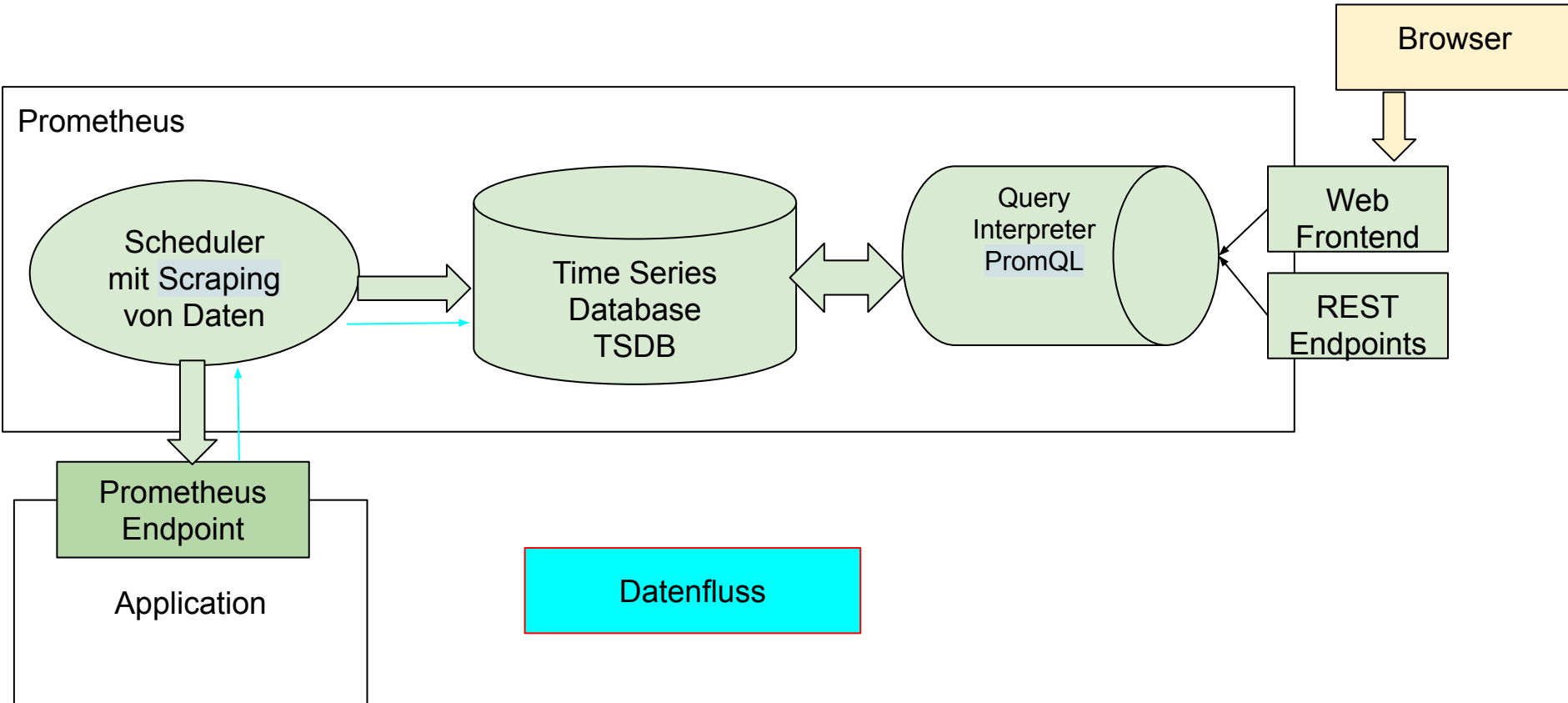
- Jedes System muss einen http-Endpoint bereitstellen, über den in einem Prometheus-unterstützten Format die Metriken bereitgestellt werden
 - Offener Punkt: Wie wird das konkret realisiert?
 - Das sieht mir nach enormen Aufwand aus, da die Anwendung angepasst werden muss
 - Lösung: Etwas später...

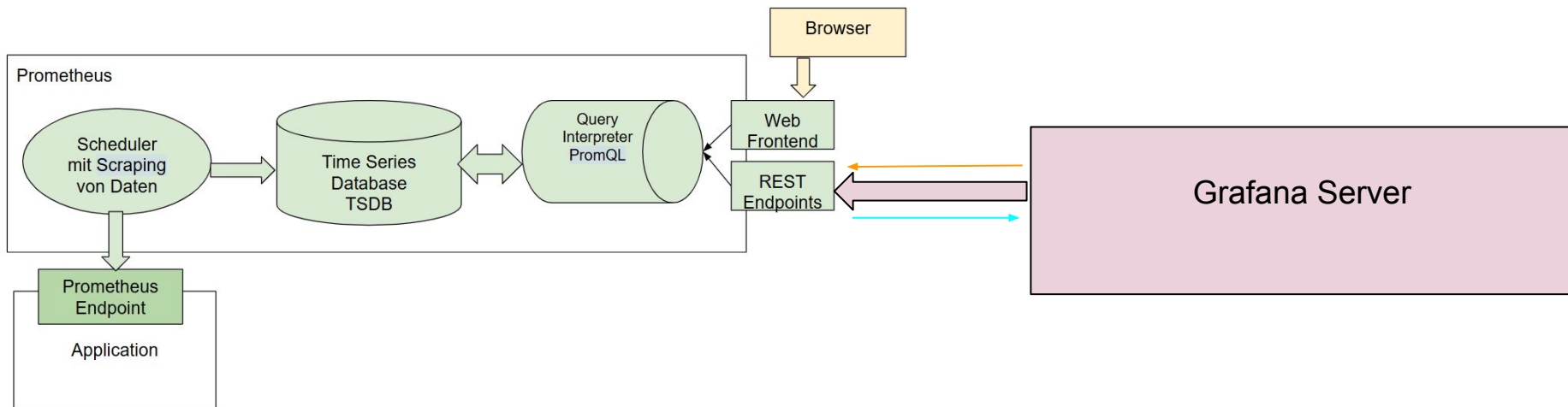


Prometheus Gesamtbild

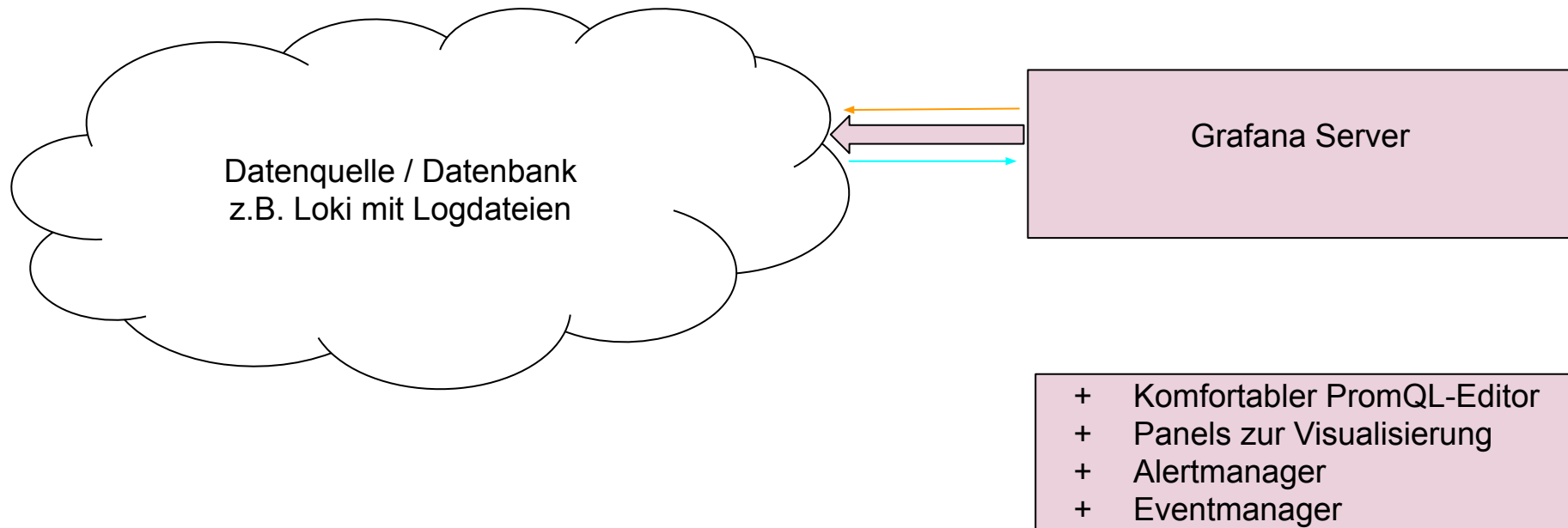


Der Prometheus Server





- + Komfortabler PromQL-Editor
- + Panels zur Visualisierung
- + Alertmanager
- + Eventmanager



Eine fertige Umgebung Teil 1

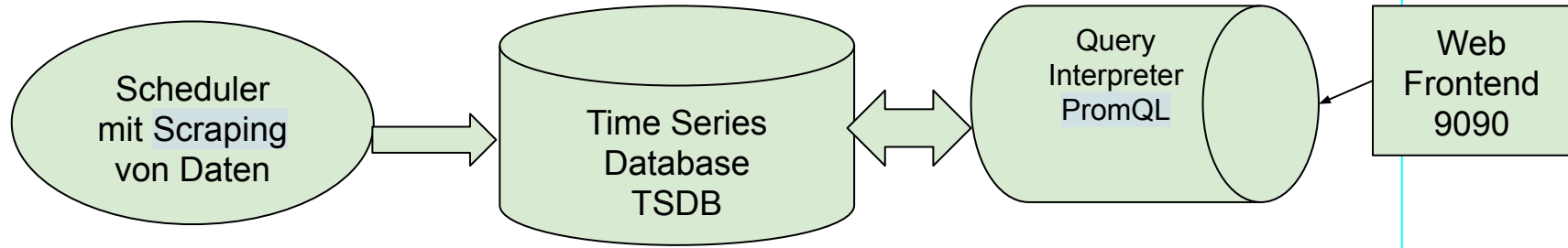
```
graph LR; Host[Host-Rechner  
Linux-Distribution  
javacream.eu] --- Endpoint[Prometheus  
Endpoint  
9100/metrics];
```

Host-Rechner
Linux-Distribution
javacream.eu

Prometheus
Endpoint
9100/metrics

Eine fertige Umgebung Teil 2

Prometheus auf javacream.eu



- Konfiguration des Prometheus-Servers
 - "Frage den Endpunkt <http://javacream.eu:9100> alle 15 Sekunden ab"

```
scrape_interval: 15s
scrape_timeout: 10s
scrape_protocols:
- OpenMetricsText1.0.0
- OpenMetricsText0.0.1
- PrometheusText1.0.0
- PrometheusText0.0.4
scrape_native_histograms: false
always_scrape_classic_histograms: false
convert_classic_histograms_to_nhcb: false
metrics_path: /metrics
scheme: http
enable_compression: true
metric_name_validation_scheme: utf8
metric_name_escaping_scheme: allow-utf-8
extra_scrape_metrics: false
follow_redirects: true
enable_http2: true
static_configs:
- targets:
  - javacream.eu:9100
```



- In der Frequenz des Scraping-Intervals werden die Metrik-Seiten aufgerufen und die Ergebnisse in der TSDB abgelegt
- Dies erfolgt in einer statischen Konfiguration unter Angabe einer URL und eines Job-Namens
 - Jeder Metrik-Identifizier wird automatisch mit den Labels job und instance ergänzt
- Hinweis
 - Eine dynamische Konfiguration ist für Anwendungen in einer Orchestrierung notwendig

- Metrik bisher
 - Identifier
 - Label
 - Wert
 - Einheit
- Zusätzlich
 - Zeitstempel, der Zeitpunkt des Scrape-Vorgangs initiiert über Prometheus
 - Typ-Information
 - Damit besteht eine Metrik gegebenenfalls aus mehreren zusammengehörigen Identifier / Label

- Counter
 - Eine Einzelmetrik
 - Wert kann nur größer werden
 - Ganzzahlen, Fließkommazahlen sind aber auch möglich
- Gauge
 - Eine Einzelmetrik
 - kann einen beliebigen Wert annehmen

Neben den Einzelmetriken gibt es auch zusammengesetzte Metriken, z.B. ein Histogramm/eine Summary

- Beispiel: Web Server
 - Anzahl der verarbeiteten Requests insgesamt
 - Anzahl der übertragenen Daten der letzten 5 Minuten
- Wie werden diese Metriken vom Typ her abgebildet, Counter oder Gauge?
 - Was muss sich das System merken?
 - Was steht in der Prometheus-Datenbank?
 - Wie werden sich die Metriken bei einem Serverneustart verhalten?

- Die Anzahl der verarbeiteten Requests ist eine fortlaufende Nummer
 - Damit ein Counter
- Nun zu den übertragenen Daten der letzten 5 Minuten
 - Es schaut so aus, als wäre das eine Gauge, da sich dieser Wert ja nach oben und unten verändern wird
 - Die Berechnung dieses Wertes kann die Applikation machen
 - Was passiert aber, wenn bei der Analyse eine andere Mittelung erfolgen soll, z.B. alle 10 Minuten
 - Das passt nicht!
 - Es ist auch möglich, das über einen Counter-Wert zu machen
 - Einfach in der Applikation eine einzige Variable merken, die hochgezählt wird
 - Die Mittelung erfolgt dann über die in Prometheus erfassten Werte

- Counter vs. Gauge bei Neustart
 - Counter ist trivial, er beginnt beim Neustart einfach bei Null
 - Gauge ist viel komplexer, der Prozess müsste den letzten Wert immer speichern, um dann korrekt bestimmt zu werden
- In der Auswertung sind Counter Reset-sicher!



- Histogramm
 - Ein Metrik-Aggregat
 - Verteilung der Werte auf verschiedene Buckets
 - Gesamtanzahl sowie die Summe der Messwerte wird ebenfalls bestimmt
- Summary
 - Bestimmung der Perzentils

- Counter
 - "Wie oft"
 - nur steigend, Reset-sicher
- Gauge
 - "Aktueller Wert"
- Histogram
 - "Verteilung der Messwerte in einzelne Bereiche / Buckets"
- Summary
 - "Verteilung der Messwerte in die "Percentiles"
 - Inkludiert zusätzlich die Summe (Counter) die Anzahl der Meßwerte (Counter)
 - Hinweis: Das funktioniert nicht, falls z.B . Knoten eines Clusters verwendet werden

- Das Scraping-Intervall macht Prometheus blind im Zeitfenster zwischen den Scrapes
 - Prometheus ist keine Echtzeit-Überwachung
- Insbesondere bei Gauges
 - Auch das führt dazu, dass Counter bevorzugt werden sollten

Erster Einstieg in die PromQL

- Ressourcen
 - Cheat Sheet
 - Prometheus Doc
 - <https://prometheus.io/docs/prometheus/latest/querying/basics/>
- KI-Unterstützung
 - gut
 - Halluzinationen sind natürlich möglich

- Einzelergebnis als Skalar (Zahl) oder String (Zeichenkette, als Metrik sehr selten)
- Instant Vector
 - Ergebnis von Metriken zu einem bestimmten Zeitpunkt
 - Jeweils der letzte Scrape-Zeitpunkt
 - Das Ergebnis einer Selektion einer Metrik
- Range Vector
 - Reihe von Werten über einen bestimmten Zeitraum
 - Selektion + Angabe des Zeitfensters der Messdaten in eckigen Klammern
 - [5m]

- Selektionsausdrücke nur mit Labels
 - Name + Label der Metrik
 - Ergebnis ist ein Instant Vector mit Werten
- Interpretation eines Instant-Vectors + dessen zeitlicher Verlauf kann in einem Graphen dargestellt werden
 - Gauge-Wert wird häufig so visualisiert
 - Welche Werte hatte z.B. die Speicherauslastung?
- Ein Graph aus einem Counter hingegen ist eher schwierig zu interpretieren
 - Der konkrete Wert eines Counters ist eigentlich uninteressant, mich interessieren die Änderungen
 - Die Änderungen ist die Steigung des Counters

- Wie hat sich eine Counter-Variable im Schnitt verändert
 - Im mathematischen Sinn also die Steigung eines Counters
- Funktionen
 - `rate`
 - Nimm den ersten und letzten Wert eines Range Vectors und teile diesen durch den Zeitraum
 - Normiere anschließend auf "pro Sekunde"
 - `irate`
 - Nimm den letzten und vorletzten Wert eines Range Vectors
 - Sonst wie `rate`

- Basieren auf dem Namen einer Metrik
 - Best Practices
 - <https://prometheus.io/docs/practices/naming/>
- Labels erweitern den Namen durch eine Liste von key=value-Paaren
 - {name=value, }
 - Hinweis
 - Eine PromQL kann auf nicht-existierende Labels selektieren
 - keine Schema-Prüfung mit Resultat, Ergebnis ist ein leeres Resultat
- Eine Selektion nur mit Labels ist möglich

- Ergebnisse können mit Vergleichsoperatoren gefiltert werden
 - `>`, `<`, `>=`, `<=`, `!=`
- Matchers
 - `=` Gleichheit
 - `!=`
 - `=~`, `!~` reguläre Ausdrücke
- Logische Ausdrücke, `|`, `&`

- up (Prometheus-interne Metrik)
 - 0
 - der letzte Scrape war nicht erfolgreich
 - 1
 - der letzte Scrape war erfolgreich
- Konfigurationseinstellung
 - Metrik vom Typ gauge, die als Wert die Konfigurationseinstellung hat
 - die Konfiguration wird durch Labels abgebildet
- info (dafür keinen Standard-Namen, bei uns `node_exporter_build_info`)
 - z.B. Versions-Tag, ebenfalls als Label

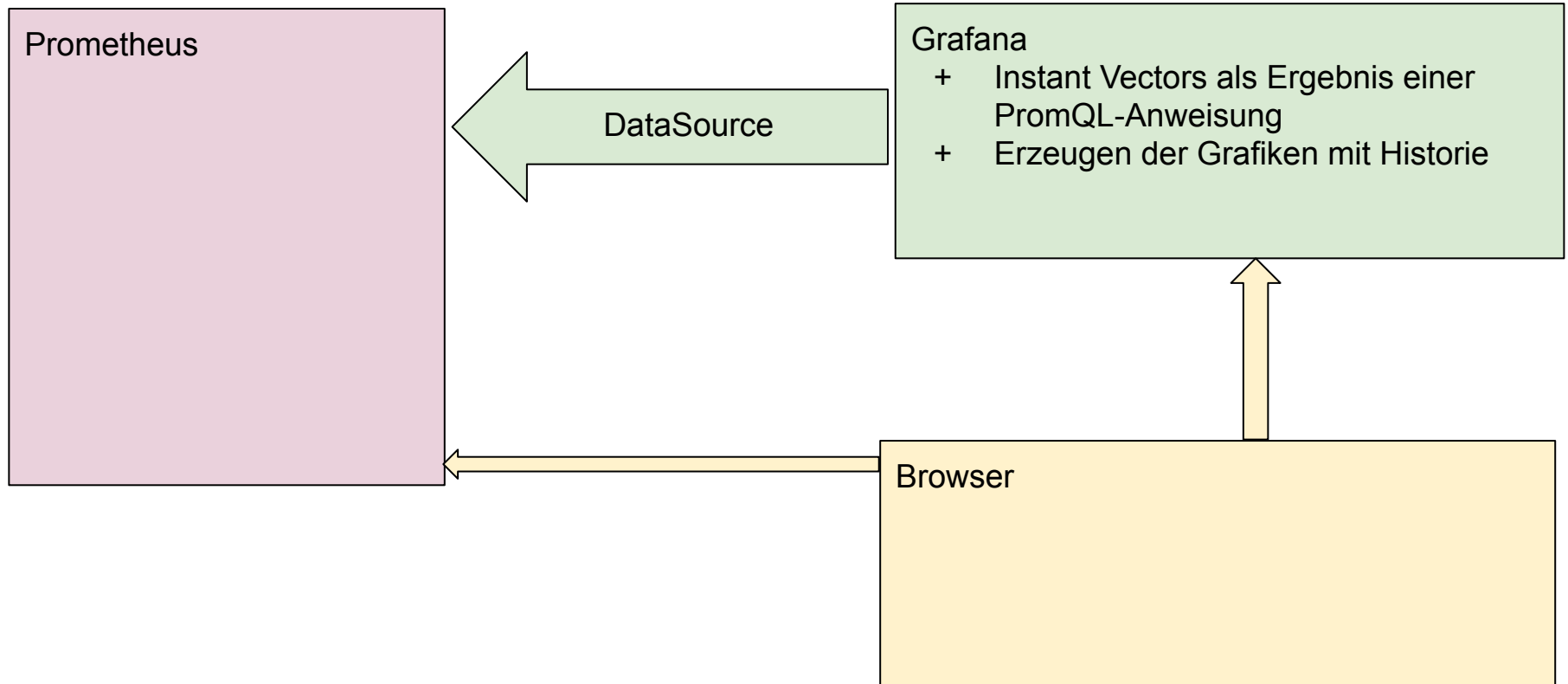
- sum, min, max, avg, count, stddev, stdvar,
- topk, bottom

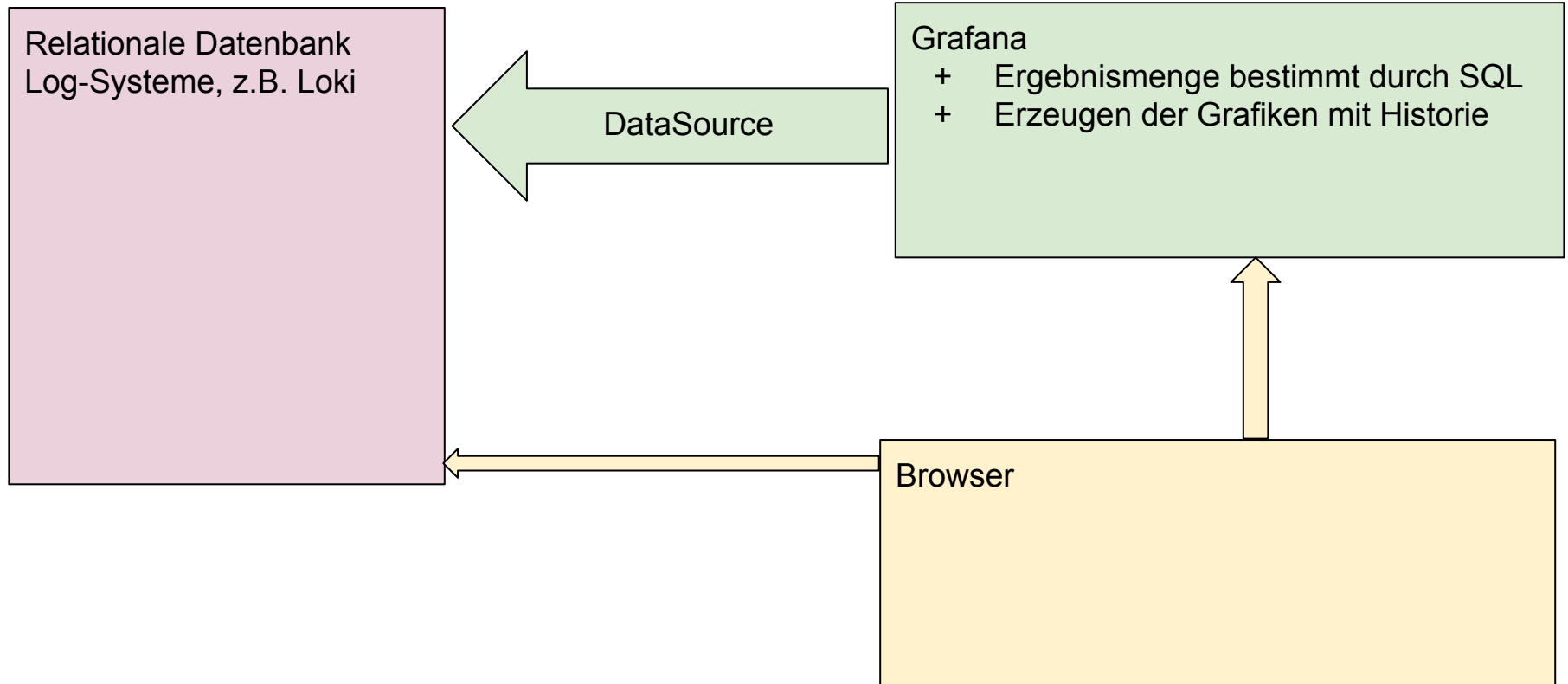
- PromQL unterstützt
 - Mathematische Operationen zwischen Instant Vector und Skalaren (= Zahlenwerten)
 - Mathematische Operationen zwischen Instant Vectors

- Summiere die idle-Zeiten aller CPUs
- Berechne den Quotienten user/idle
- Berechne den Quotienten als Prozentzahl
- ...

Grafana - Kurzeinführung

- Zugriff
 - javacream.eu:9092
- Credentials
 - admin
 - admin123





- Connections / Data Source
 - die zu visualisierende Datenquelle
 - Prometheus ist eine unterstützte Datenquelle
- Panel
 - Visualisierung eines Instant Vectors gebildet auf Basis einer PromQL
 - “Unikat”-Panels, das sind die normalen Panels
 - Library Panels sind vergleichbar ,mit einer Klasse / einem Template
 - Änderungen innerhalb eines Library Panels werden auf alle konkreten Panels übertragen
- Dashboard
 - Aggregat von Panels
 - Gruppiert in einzelne Zeilen / Rows

- Dashboards
 - Snapshots
 - “Einfrieren” eines Dashboards
 - Zugriff erfolgt über URL
 - Download als Image
 - Playlist
 - Abspielen verschiedener Dashboards innerhalb der Grafana Applikation
- Import
 - -> nächste Seite

- Ein Dashboard wird immer repräsentiert durch eine Datei = Quellcode
- Bezug zu Versionsverwaltung
 - Grafana bietet eine integrierte Versionsverwaltung für Dashboards
 - Bei jeder Änderung wird eine Möglichkeit angeboten, die Änderung zu beschreiben
 - Historie der Änderungen ist damit verfügbar
 - Hinweis
 - Selbstverständlich können Dashboards auch in einem Git-Repository verwaltet werden
- Die Code-Repräsentierung eines Dashboards kann jederzeit angezeigt werden
 - Sehr technisch
- Der Code des Dashboards kann exportiert werden
 - Format: JSON

- Eine Sammlung von Dashboards as Code, die anderen zur Verfügung gestellt werden können
 - Ablage der exportierten Dashboards
- Dashboards können aus der Registry importiert werden
- <https://grafana.com/grafana/dashboards/>

- Als eine Datei, genauer: Eine Quellcode-Datei, noch genauer: Eine Quellcode-Datei im JSON-Format
 - “Zusammenklicken” eines Dashboards ist im Endeffekt Programmierung
- Dashboards werden selbstverständlich in einem SCM abgelegt
 - z.B. im Git-Servers
 - Beim Aufsetzen eines Grafana-Servers wird das Dashboard-Verzeichnis als Repository gecloned
- Community-Dashboards können genutzt werden
 - Öffentliches Dashboard Repository
 - <https://grafana.com/grafana/dashboards/>

- "Spickzettel" des Node Exporter Full-Dashboards
- Befüllen Sie jeweils ihr leeres Panel mit
 - Einem Panel-Gauge mit "CPU Busy"
 - Time Series "CPU Busy"
 - Time Series mit CPU modus="idle"
- Übersicht der Galerie der vorhandenen Panel-Typen
- Settings

Hinweis: Statt der Variablen werden im leeren Panel die Werte für instance und job und rate_interval hart eingegeben

- Mit Variablen werden Dashboards wiederverwendbar
 - Best Practice
 - Queries von Panels sollen keine fixen Werte enthalten, sondern sich immer auf Variable beziehen
- Definition von Variablen in den Settings des Dashboards
 - Typen
 - Freitext
 - Query
 - PromQL-Ausdruck, der eine Liste von Werten liefert
 - Interval
 - Zeitangabe, z.B. für rate
 - Custom
 - Auflistung von Werten, die dem Benutzer zur Auswahl angeboten werden
 - ...

- Allgemein
 - `$__from` Startzeit des aktuellen Zeitbereichs (Unix ms)
 - `$__to` Endzeit des aktuellen Zeitbereichs (Unix ms)
 - `$__range` Zeitbereich (z. B. 1h)
 - `$__range_ms` Zeitbereich in Millisekunden
 - `$__range_s` Zeitbereich in Sekunden
 - `$__interval` Automatisch berechnetes Intervall
 - `$__interval_ms` Intervall in Millisekunden
 - `$__rate_interval` Empfohlenes Intervall für `rate()` (Prometheus)
 - `$__timezone` Zeitzone des Dashboards
 - `$__dashboard` Name des Dashboards
 - `$__org` Organisations-ID
 - `$__user`
- Bezug zu Prometheus
 - `$job`
 - `$instance`
 - `$device`
 - `$mountpoint`
 - `$cpu`
 - `$interval`

- Einführen von Variablen im "leeren" Dashboard
 - node als Query-Variable
 - job als Custom-Variable mit den Auswahlmöglichkeiten "Linux Node" und "Non Existent"
 - Interval-Variable zur Bestimmung des Rate-Intervalls

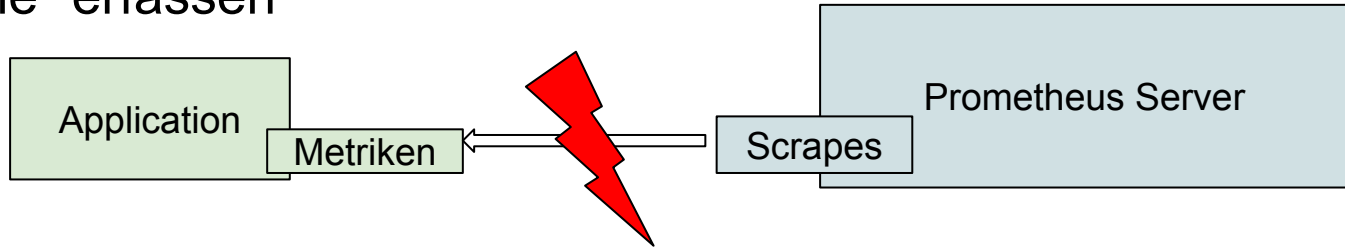
Aufsetzen eines Prometheus/Grafana Stacks

- Nun die folgende Befehlssequenz eingeben
 - `cd prometheus`
 - `cd server`
 - `docker compose up -d`

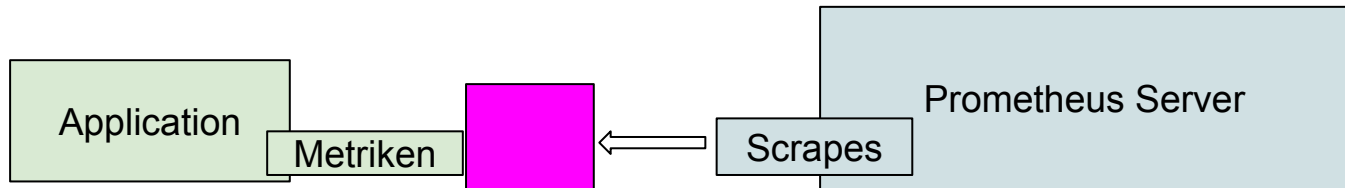
CHECK im Browser

- <http://localhost:9090>
 - zeigt das Prometheus-Frontend
- <http://localhost:9092>
 - zeigt das Grafana-Frontend
 - mit eingerichteter Data Source Prometheus
- Im Prometheus-Frontend
 - `up`
 - pushgateway, prometheus und Linux Node sind auf 1

- Ausgangssituation ist, dass die meisten Anwendung Metriken "irgendwie" erfassen



- Dieses "irgendwie" wird durch einen Prometheus **Exporter** kompatibel zu Prometheus gemacht



- Einbinden des javacream.eu:9113-Exporters
 - dieser stellt Web Server Metriken bereits
 - Zugriff den Web Server über javacream.eu:8080
- Check
 - up zeigt einen neuen laufenden Prozess
 - `nginx_http_requests_total` ist als Metrik vorhanden
- Erstellen (?) Sie ein repräsentatives Dashboard, das die Web Server Metriken darstellt
- Hinweis
 - Falls Sie den Prometheus-Server durchstarten wollen, was bei jeder Änderung der `prometheus.yml` notwendig ist
 - `docker compose down`
 - `docker compose up -d`

im Verzeichnis
`docker/prometheus/server`

- Java ist für sehr gut geeignet
 - Die Laufzeitumgebung ("Java Virtual Machine") sammelt mit JMX eine große Menge von Metrik-Informationen
 - Der JMX-Exporter öffnet diese Metriken für Prometheus
- Noch besser ist Java mit Spring Boot
 - Rein deklarativ, Einträge in der pom.xml bzw. application.properties
- Andere Sprachen zu finden unter
 - <https://prometheus.io/docs/instrumenting/clientlibs/>

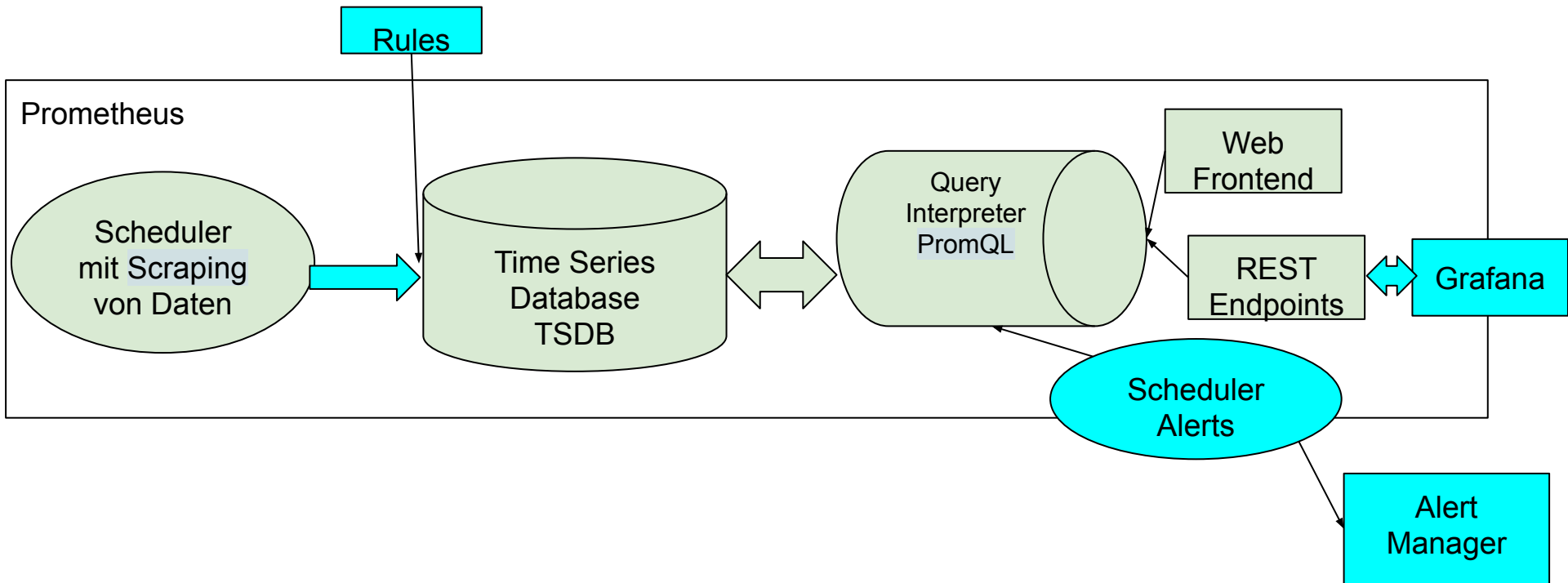
Weitere Tools

- Kurzlaufende Prozesse (z.B. Batches) sind für Prometheus unhandlich in der Überwachung
 - Kann tatsächlich beendet sein, bevor Prometheus scraped
 - die Zeitreihen haben Lücken
 - up = 0
 - ...
- Idee
 - Statt die kurzlaufenden Prozesse von Prometheus zu scrapen wird umgestellt auf einen Push-Mechanismus
 - Dazu wird das Pushgateway verwendet
 - ein separat laufender Prozess, der von Prometheus überwacht wird und für laufende Prozesse Metriken bereitstellt

- Prometheus Alertmanager
- Kibana
- Splunk On-Call
- Grafana Alertmanager

- Alerting = Digitalisierte und automatisierte Überwachung
- Bestandteile
 - Alert Rule
 - Regelwerk: Alarmsituation ist vorhanden
 - Contact Point
 - Wem wird der Alert gemeldet?
 - Person oder Gruppe identifiziert über eMail, WhatsApp, ...
 - Externes Alertsystem
 - Notification Policy
- Details
 - Alert Rule = Bedingung, die aber in einem bestimmten Zeitraum erfüllt sein muss
 - Silencing
 - Testmöglichkeit muss vorhanden sein
 - Zusammenfassung zusammengehöriger Alert-Kaskaden

- Prometheus-Alerts
 - PromQL-Anweisung ALERTS
 - Wie eine Metrik haben die ALERTS eine Zeitreihe / Historie
- In Grafana gibt es das Panel "Alert List", das die aktuellen Alerts darstellt
 - Grafana unterstützt keine Historie



Prometheus Exporter im Detail

- Nun die folgende Befehlssequenz eingeben
 - `cd ..`
 - `cd node_exporter`
 - `docker compose up -d`

CHECK im Browser

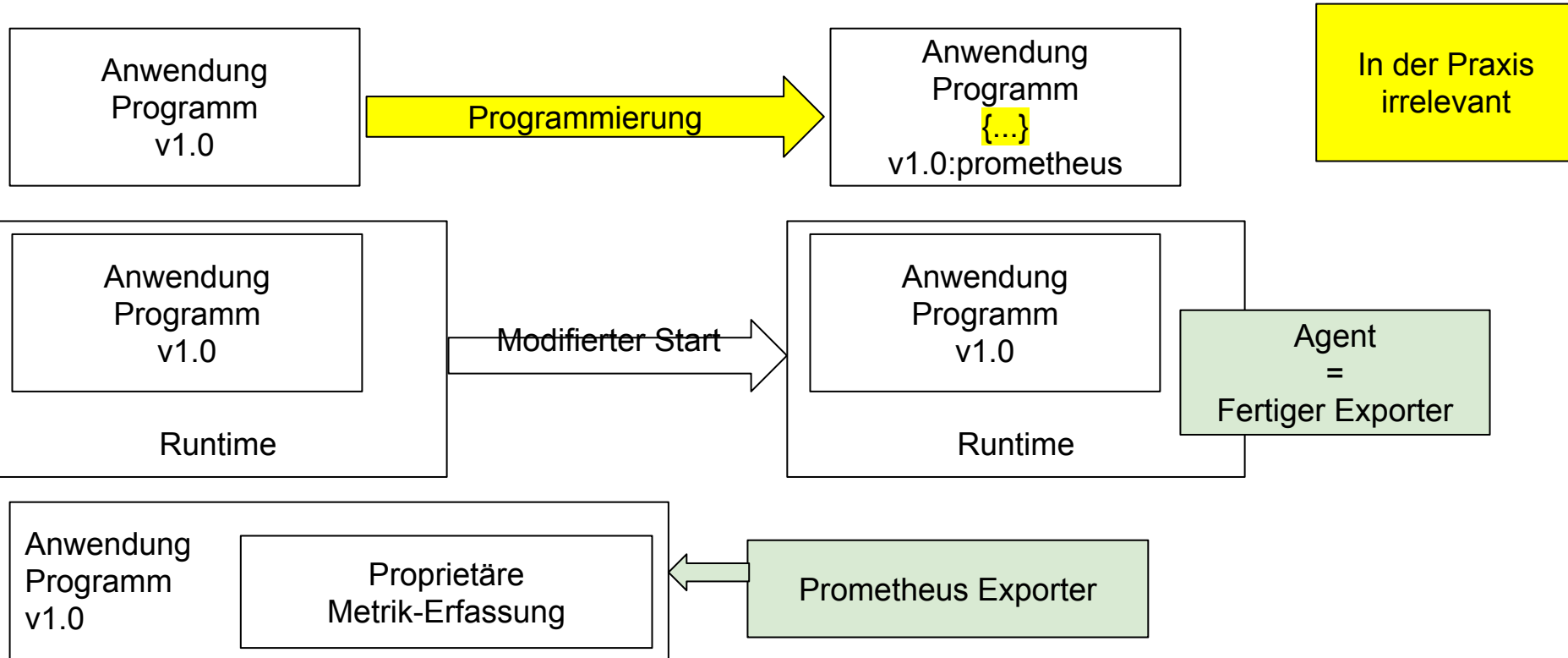
- <http://localhost:9091/metrics>
 - zeigt wieder Linux-Metriken
- Im Prometheus-Frontend
 - up
 - auch Linux Deskmate ist nun auf 1

- Stellt einen Prometheus-Metrik-Endpoint zur Verfügung
- Die interne Implementierung weiß, wie sie die Informationen aus einer laufenden Anwendung ziehen kann
- Es existiert eine Palette fertiger Exporter-Implementierungen
 - Prometheus-Community
 - Herstellern von Produkten
 - Open Source Community
- Eigene Exporters können bei Bedarf natürlich jederzeit programmiert werden
 - Im Wesentlichen genügt hierfür die Bereitstellung eines Web Servers

- Stoppen Sie ihr Prometheus/Grafana
 - docker compose down
 - im Verzeichnis "server"
 - Ändern Sie die Konfiguration um einen weiteren Job, in dem Sie Ihren Deskmate eintragen
 - ip a
 - docker compose up -d
 - Starten des Node Exporters auf Ihrem Deskmate
 - cd ..
 - cd node_exporter
 - docker compose up -d
 - Zusätzlich: Scraping-Konfiguration für den nginx-Exporter
 - Auf Referenten-Deskmate unter 9113
 - Dafür bitte ein Dashboard konzipieren/erstellen/**importieren&anpassen**

Hatte ich vergessen...

Kategorien von Exportern



- Java Anwendungen laufen in der Virtual Machine
- Durch den Java Exporter werden Prometheus-Metriken aus der JVM bereitgestellt

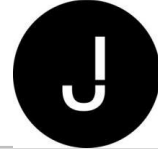
- Als Beispiel hierzu haben wir bisher schon den Node Exporter genutzt

- Java Management Extension, JMX steht für jeden Java-Prozess sofort zur Verfügung
 - Damit sind Standard-Metriken des Java-Prozesses sofort verfügbar
 - `java.lang:type=Memory`
- JMX und Überwachung passt nur scheinbar gut zusammen
 - JMX ist nicht nur Überwachung, sondern auch Administration
 - Über einen JMX-Client könnte beispielsweise eine Garbage Collection angestoßen werden

- Dieser liest aus der Java-Anwendung die in JMX bereitgestellten Informationen und stellt diese Prometheus-konform zur Verfügung
- Existiert in 2 Ausprägungen
 - Klassischer Standalone Exporter
 - selten verwendet
 - Java Agent

- Der Actuator ist für Erfassung und Bereitstellung von Metriken verfügbar
- Der prometheus-Endpoint kann konfigurativ aktiviert werden und stellt dann unter `host:port/actuator/prometheus` ausgewählte JVM-Metriken zur Verfügung

- Bitte Namenkonventionen beachten
 - Spring Boot "toleriert hier einiges"
- Wahl des richtigen Metrik-Typs
 - Counter wo immer es irgendwie geht
 - Gauge
 - Echte beliebige Werte
 - Konfigurationseinstellungen
 - Pseudometrik
 - Dient zum Transport von Labels
- Den Einsatz "ob überhaupt" überdenken



Dies und Das

- Umgang mit Zuständen
 - Im Endeffekt eine Enumeration
 - Abbildung auf Zahlen
 - Vorsicht: STARTING, STOPPING, RUNNING -> 0,1,2
 - Abfrage nach diesem Status ergibt eventuell als Ergebnis 1.45 -> Seite 46
 - Logische Werte sind immer 0 (false) 1 (true)
 - Kompatibel mit den Ergebnis-Vektoren eines bool-Vergleichs
- Umgang mit textuellen Informationen
 - Metrik mit der Endung _info
- Das “Measurement-**API**” ist ernst zu nehmen
 - Hinzufügen oder Entfernen von Labels zu einer Metrik wird eine Anpassung der PromQL-Statements erfordern

- Lösung
 - `last_over_time(range_vector)`
- Alternative: Label Patterns “enum”
 - `{state="RUNNING|STARTING|STOPPING"} 0|1`

- Nur sehr rudimentär
 - `predict_linear`
- Ernsthafte statistische Analysen = Machine Learning nutzt Prometheus nur als Datenquelle

- Externe “Programme”, die in Prometheus als Quellcode hinterlegt werden
- Diese Programme werden nicht interaktiv aufgerufen, sondern werden im Hintergrund von Prometheus ausgeführt
 - Die Ergebnisse stehen dann zur Verfügung

- Bisher
 - statische Konfiguration der Scrapes
- Problem
 - Funktioniert nur sehr umständlich in einer orchestrierten Cloud-Umgebung
- Lösung
 - Prometheus ist kompatibel zu z.B. Kubernetes
 - Eine Konfiguration eines Scrapes erfolgt durch Angabe einer Service-Registry
 - diese “weiß”, wie viele Container aktuell laufen
 - In der Praxis wird Prometheus in Kubernetes integriert sein

- Node Exporter
 - eigentlich ist das ein “Host-Exporter”
- Container-Unterstützung
 - <https://prometheus.io/docs/guides/cadvisor/>